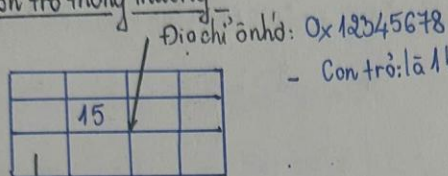


II / C Embedded (Pointer, Bitwise, Typedef, Macro, Enum, Struct, Union, Pack)

1/ Pointer (con trỏ thông thường, con trỏ void, con trỏ hàm)

- C quan trọng vs Embedded vì bộ nhớ, ram của vdk, vxl rất nhỏ nên sd pointer để quản lý memory rất quan trọng.

a) Con trỏ thông thường



- Con trỏ: là 1 biến quản lý gtrị và gtrị đó là địa chỉ của 1 ô nhớ $\neq$

Ô nhớ: gồm { gtr ô nhớ: (15)
địa chỉ ô nhớ: (0x12345678)

Xem thêm trong code.

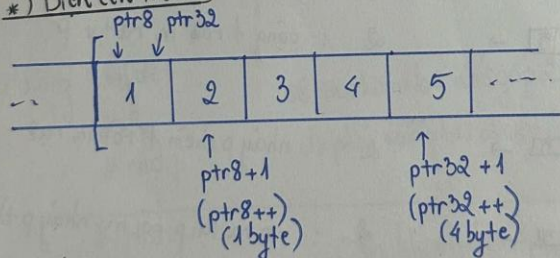
- Quản lý memory giữa có/ko dùng con trỏ: ko phụ E vào kiểu hàm, struct, ... mà kthức của *ptr lun là 8 byte (x64); 4 byte (x86)

$\Rightarrow$ Sd pointer sẽ ít tốn bộ nhớ.

32 bit : quản lý tới đa $2^{32} \sim 4GB$.

1 byte 8bit lun đc 0x00 $\rightarrow$ 0xFF

*) Dịch con trỏ



b) Void Pointer

- Dùng để truyền 1 pointer bk mà ko quan tâm kiểu dữ liệu của nó
- $\Rightarrow$ Sau đó tùy project mà ép kiểu mình mong muốn

c) Function Pointer

- 1 Function khi đc khởi tạo sẽ có 1 địa chỉ
- + Call qua name (hàm)
- + Call qua địa chỉ (address function)

BT Point:

4, 5, 3, 2, 2, 2

Đề

```
int main() {
```

```
    int arr[] = {1, 2, 3, 4, 5}
```

```
    int *ptr = arr;
```

```
    printf("%d %d %d %d %d %d", *ptr, ++*ptr++, (*ptr)++, *ptr++, ++*ptr, ++*ptr)
```

```
    return 0;
```

1 2

←

```
}
```

*ptr = 1

(ptr → 1), *ptr = 1 (0)

++*ptr++

(*ptr = 3) *ptr = 3 (0)

4, 5, 3, 2, 2, 2

Hướng dẫn:

1	2	3	4	5
---	---	---	---	---

(int)

2 2 3 4 5 ptr int

* Có 2 lưu ý: Phải từng viết hàm printf thì mới làm được. (B2: RTOS)

Cơ chế printf: thực hiện từ phải qua trái (xử lý từ phải sang trái nhưng print ra vẫn theo thứ tự ngược viết)

++*ptr

2	2	3	4	5
---	---	---	---	---

 → 2 : cộng 1 rồi in ra tại p

*++ptr

2	2	3	4	5
---	---	---	---	---

 → 2 : nhảy p thêm 1 rồi in ra tại

*ptr++

2	2	3	4	5
---	---	---	---	---

 → 2 : in ra tại p rồi mới nhảy p thêm 1

(*ptr)++

2	2	3	4	5
---	---	---	---	---

 → 3 : in ra tại p rồi mới tăng giá trị tại p thêm 1

*++*ptr++

2	2	3	4	5
---	---	---	---	---

 → 5 : tăng giá trị tại p rồi mới nhảy p thêm 1

*ptr

2	2	3	4	5
---	---	---	---	---

 → 4 : in giá trị tại p

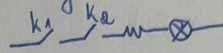
2/ Typedef

- Tạo 1 tên ms cho 1 kiểu dữ liệu.

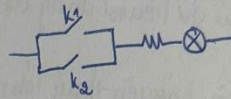
3/ Bitwise

- VĐK: memory rất hạn chế $\Rightarrow$ quản lý rất quan trọng

+ And bit (&) $\Rightarrow$ Clear bit $\rightarrow 0$



+ Or bit (|) $\Rightarrow$ Set bit $\rightarrow 1$



+ Not bit (~) $\Rightarrow 1 \rightarrow 0, 0 \rightarrow 1$

+ Xor bit giống nhau 0, ngược lại

+ Dịch bit (<<, >>)

0	0	0
0	1	0
1	0	0
1	1	1
0	0	0
0	1	1
0	0	1
1	1	1

4/ Macro

- Tiền xử lý: sẽ thực hiện trc compiler
(#include, #define, #ifdef)

khi dùng #ifdef ---
#else ---
#endif

sẽ tiết kiệm bộ nhớ gộp 2 lần so vs viết if trong main
do phải cấp phát địa chỉ cho cả 2 function

#ifndef

#define

#endif

main.h

abcd

stm32.c

#include "main.h"

hcr... .c

#include "main.h"

$\Rightarrow$ khi chạy nó phân thêm 2 lần main.h

(abcd x 2 $\Rightarrow$ phi)

Chỉn khi #ifdef trong stm32.c thì nó sẽ ktra
nếu add rồi nó sẽ ko add nữa để tồn memory.

5/ Enum

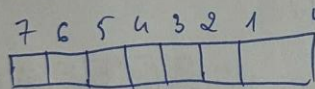
- Enum định nghĩa các case trạng thái (states)
- Mặc định ko gán thì pt đầu tiên là 0
→ pt tiếp theo là 1 (+1)

6/ Struct.

- Tự tạo kiểu dữ liệu (do người dùng tự định nghĩa)
- Kiểu dữ liệu nguyên thủy lớn nhất → size cơ sở
⇒ block của nó phải đạt độ 4 byte

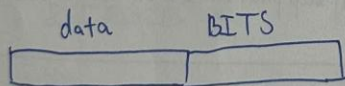
7/ Union

- Tương tự Struct: tiết kiệm bộ nhớ.
- Xử lý từng bit 1 trong thanh ghi
- ko share dữ liệu



BITS : truy cập từng bit thanh ghi
union

data : truy cập cả thanh ghi



struct.

8/ Extern

- Sử dụng biến x trong 1 file.c ≠ trong main.c

↓
cái như 1 module (ko phải file header)

sử dụng extern int x; (trong main.c)

- extern giúp ... hiểu là biến x đã đc khai báo ở đâu đó trong memory rồi nên cứ sd đi

⇒ Trong nhưng sẽ ko dùng vì sẽ phải extern nhiều.

⇒ Mình sẽ tạo lib.h: và extern trong đó rồi ở file.c nào cần thì #include nó vào

9/ Volatile

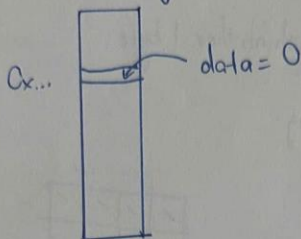
STM32



Cấp phát động



Phân vùng Ram (Heap, Stack)



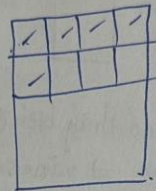
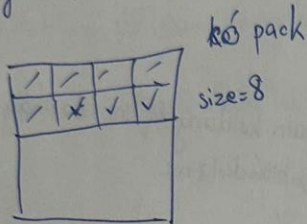
- Compiler sẽ tối ưu nếu nó thấy biến đó trong main ko dùng ts hoặc ko thay đổi nên nó sẽ xóa luôn để tiết kiệm $\Rightarrow$ biến sẽ ko đọc lại biến data nx
- Tuy nhiên có thể biến data nó thay đổi trong ngắt
(Ngắt là 1 sk mà ng dùng muốn CPU dừng lại đột ngột để xử lý function của mình)
 $\downarrow$ thay đổi đột ngột và trong thanh ghi
 $\Rightarrow$ Khi dùng volatile sẽ cho compiler biết là biến này sẽ đc dùng trong n th thay đổi
 $\Rightarrow$ compiler sẽ để biến đó trong ram và vẫn đọc lại liên tục mà ko reset nó mất.

10/ Set, clear, check bit

13/ Weak (Bài 8 - STM32 Basic)

11/ Pack

- Kiểu dữ liệu cỡ số.
- Struct: căn theo cái trường dữ liệu có kiểu dữ liệu to nhất
- # pragma pack (push, 1) : căn chỉnh theo 1 byte.
- # pragma pack (pop) : ^(byte)đứng.

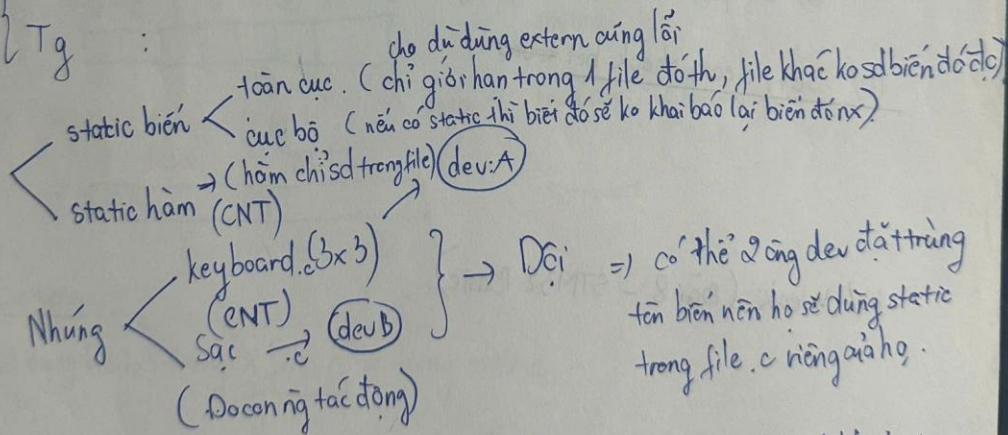


Cỡ pack
Size = 5

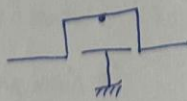
⇒ Mục đích tránh lãng phí memory

12/ Static

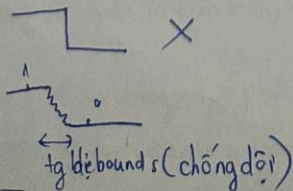
Ko gian :
Tg :



MCU
CPU hiệu 1
0



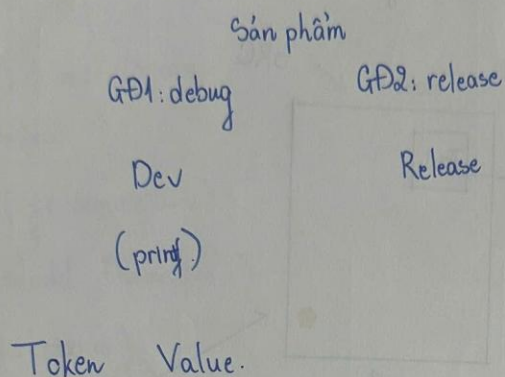
khi nhấn bị rung động 1 khoảng + gây nhiễu ký ngẫu nhiên.



Bài 19: Thực tế - Bật/tắt debug log.

* Bộ nhớ

- printf: tốn bộ nhớ
- version



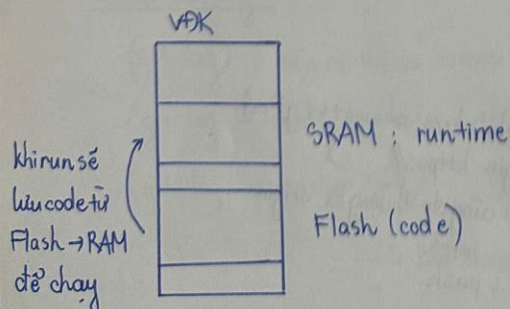
⇒ Tạo 1 cái file include.h để ... (code)
bỏ lệnh printf

* Const: ~~Si~~ ns vs compiler: " cái này là hằng, đừng cho ai sửa, để trong Flash

- Số cho các giá trị cố định ko đổi khi run code
(bảng mã, hằng số, ...)

- Const để đặt trong ROM, ko chiếm dung lượng của RAM.
(Flash)

⇒ Giúp an toàn và tiết kiệm RAM



Vì RAM rất ít

⇒ Const sẽ lưu vào Flash (nếu biến ko đổi giá trị)
mảng

TEST C EMBEDDED – HỌC CÙNG ET



**Lưu ý: Bài thi chỉ có giá trị khi bạn tuân thủ các quy tắc sau*

- Không sử dụng IDE
- Không sử dụng máy tính trên laptop, casio,...
- Làm bằng chính khả năng C embedded của mình, không search google khi chưa được phép

Q1.

```
int i;  
for (i = 1; i < 101; ++i)  
{  
    i++;  
}  
  
printf("%d", i);
```

Q2.

```
char i;  
for (i = 0; i < 128;)  
{  
    i++;  
}  
printf("%d", i);
```


Q3.

```
typedef union
{
    struct Register
    {
        unsigned int value1 : 3;
        unsigned int value2 : 2;
        unsigned int value3 : 3;
    } value;
    unsigned int raw;
} DemoRegister;

DemoRegister demo_reg;
demo_reg.value.value1 = 1;
demo_reg.value.value2 = 2;
demo_reg.value.value3 = 3;

demo_reg.raw = 0xFF;
printf("%d %d %d %d", demo_reg.value.value1, demo_reg.value.value2,
demo_reg.value.value3, demo_reg.raw);
```

Q4.

```
void hello()
{
    printf("Hello");
}
void welcome()
{
    printf("welcome");
}

int main()
{
    void (*funcPtr)();
    void (*funcPtr2)();

    funcPtr = hello;
    funcPtr2 = welcome;
    funcPtr = funcPtr2;

    funcPtr();
}
```

Q5.

```
#define HCET -1
#ifdef HCET
void printMes()
{
    printf("WIN");
}
#endif

#ifndef HCET
void printMes()
{
    printf("LOSE");
}
#endif

int main()
{
    printMes();
}
```

Q6.

```
enum number
{ one = 1,
  two = 2,
  three,
  five,
  four = 4,
  six,
  seven = 7
};
int main()
{
    printf("%d %d %d", five, four, six);
}
```


Q7.

```
int a = 1, b = 2, c = 3, d = 4;
int *pa = &a, *pb = &b, *pc = &c, *pd = &d;
pb = &c;
pb = &d;
pa = &b;
pc = &a;
b = a + d;
d = b + c;
pb = pc;
pb = pa;
pa = &b;
pc = pd;
pb = &c;
pd = &d;
printf("%d %d %d %d", *pa, *pb, *pc, *pd);
```

Q8.

```
int main()
{
    int arr[] = {1, 2, 3, 4, 5};
    int *ptr = arr;
    printf("%d %d %d %d %d %d", *ptr, ++*ptr++, (*ptr)++, *ptr++, *++ptr, ++*ptr);
    return 0;
}
```

Q9.

```
typedef struct
{
    int a;
    int b;
    int c;
    int d;
    int f;
    int g;
    int h;
} MyStruct;

typedef struct
{
    char a;
} MyStruct2;

int main()
{
    MyStruct a;
    a.a = 65;
    a.b = 66;
    a.c = 67;
    a.d = 68;

    MyStruct2 *b = (MyStruct2 *)&a;

    b += 4;

    printf("%c", b -> a);
}
```

Q10.

```
typedef struct
{
    short a;
    long b;
    char c;
} tem;

typedef struct
{
    short a;
    char c;
    long b;
} tem2;

int main()
{
    tem a;
    tem2 b;
    printf("%d %d", sizeof(a), sizeof(b));
}
```

END. THANKS

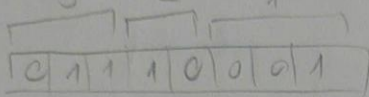
Test C Embedded cuối khóa

Câu 1: (For)

101

Câu 2: (Byte) lặp vô hạn

Câu 3: (UNION)



256

7 3 7 255

Câu 4: (Function pointer)

welcome

Câu 5: (Tiền xử lý)

WIN

Câu 6: (Enum) Số nguyên.

4 4 5

Câu 7: (Pointer)

do thực tế số nhiều lần

vì cen trở chỉ thao tác

1 địa chỉ tại 1 đ' thời

pa → &a

pb → &b → &c → &d

pc → &c → &a

pd → &d

⇒ để ý các pa, pb, pc, pd ở cuối cùng
a, b, c, d giá trị là b, c, d

b=5

d=5

→ &b

→ &a → &c

→ &d

→ &d

b=5

d = 5 + 3

1 2 3 4 5

2 2 3 4 5

↑

5 3

8 8

4 4

5 3 8 8

Câu 8:

++*ptr

(C Embedded

++*ptr 1 2 3 4 5
 2 2 3 4 5
 ↑_p

4 5 3 2 2 2 /

*++ptr 2 2 3 4 5
 ↑_p

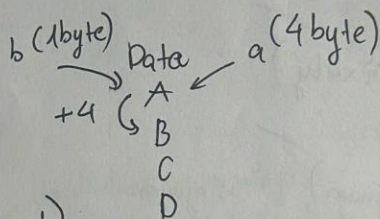
*ptr++ 2 2 3 4 5
 ↑_p

(*ptr)++ 2 2 4 4 5
 ↑_p

++*ptr++ 2 2 5 4 5
 ↑_p

Câu 9 (E_p kiểu)

B



~~Câu 10~~ (Kiểu dữ liệu - Size struct)

2 + 2
 4 12 /
 4
 2 + 1 + 1 8
 4 12
 4

